AMATH 482: HOME WORK 1

VARUN PANUGANTI

ACMS Department, University of Washington, Seattle, WA
varunp5@uw.edu

Abstract. This report studies a fully connected neural network for FashionMNIST classification. A baseline model is constructed and then modified to compare the effects of different optimizers, initialization methods, and batch normalization. Performance is evaluated using training loss, validation accuracy, and test accuracy. The results show that optimizer choice and learning rate strongly affect convergence and stability. Among the tested configurations, Adam with a smaller learning rate gives the strongest overall performance, while Kaiming initialization performs best among the initialization methods. Batch normalization improves training loss, but does not improve validation or test accuracy relative to the baseline.

1. Introduction and Overview

We are provided the FashionMNIST dataset, which contains images of fashion items. In this report, the goal is to train a working baseline classifier, and also observe how changing certain parameters will affect the performance of our classifier. The parameters which will be changed are the optimizer, the learning rate, the weight initialization, and whether or not we use batch normalization. We will also track the train loss, validation accuracy, and test accuracy so we can compare our models.

2. Theoretical Background

2.1. **Fully Connected Layers.** For this project we are conducting classification using Neural Networks. Neural Networks model hierarchical decision making, and consist of multiple layers: an input layer, several hidden layers, and an output layer. At every hidden layer, a linear transformation is conducted of the form:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

After that an Activation function is applied:

$$a^{(l)} = \sigma(z^{(l)})$$

For the final output, no activation function is applied.

$$s^{(K)} = z^{(K)} = W^{(K)} a^{(K-1)} + b^{(K)}$$

Where $z^{(l)}$ is called the preactivation. $a^{(l)}$ is the output of layer l (The letter "a" is for activation). $a^{(0)} = x$ is the input vector.

Date: March 20, 2026.

2.2. **Softmax for Output Layer.** Once we have obtained our arbitrary scores, we can convert them into probabilities over class. Assuming these are our scores produced by our model: $s = (s_1, s_2, ..., s_c) \in \mathbb{R}^c$, the Softmax function will convert these scores into probabilities:

$$softmax(s)_k = \frac{e^{s_k}}{\sum_{j=1}^{c} e^{s_j}} \qquad k = 1, ..., c$$

A better interpretation of this is: $P(Y = k) = softmax(s)_k$

2.3. **The Cross Entropy Loss and Optimization.** Cross Entropy is a useful tool we can use to penalize confident and wrong predictions. Simply put, if the model assigns high probability to the correct class $p_{y_i}(x_i; W) \approx 1$, then the loss is small, whereas if the model assigns low probability to the correct class $p_{y_i}(x_i; W) \approx 0$ then the loss is big. Our objective is to optimize the cross entropy loss:

$$L(W, b) = \frac{-1}{m} \sum_{i=1}^{m} \log(softmax_{y_i}(s(x_i, W, b)))$$

We can use gradient descent to train our neural network. η is our learning rate, and controls how large of a step we take in the direction of the negative gradient:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{dL}{dW^{(l)}} \qquad b^{(l)} \leftarrow b^{(l)} - \eta \frac{dL}{db^{(l)}}$$

In this project we use 3 different optimizers. Stochastic Gradient Descent updates parameters using only the current batch. RMSProp adapts the learning rate for each parameter by maintaining a moving average of the squared gradients. Adam keeps track of the decaying average of past gradients and past squared gradients.

2.4. **Initialization and Batch Normalization.** Initialization is the process of choosing our starting values of our weights W and biases b, before we start training the model. In this report we test 3 different initializations: Random Normal, which uses a normal distribution with small variance to assign our weights; Xavier, which scales weights based on the input and output units to keep the variance of the activation functions across layers consistent; and finally, Kaiming (He), which adjusts the variance to account for ReLU zeroing out negative values. We can also use normalization in the process of batch normalization, where we normalize the input to each layer so we can stabilize our training process.

3. Algorithm Implementation and Development

I used NumPy for numerical array operations, such as storing training loss and validation and test accuracy across each epoch [2]. I used PyTorch for loss functions, and for optimizers which were used to train the connected neural networks [4]. TorchVision was used to download the FashionMNIST dataset necessary for the assignment and for normalization [6]. I used Matplotlib to create graphs of the training loss and validation accuracy [3]. Tqdm was used to show the training bars during training [1]. Finally, I used Scikit-Learn to split the data into training and test data [5].

4. Computational Results

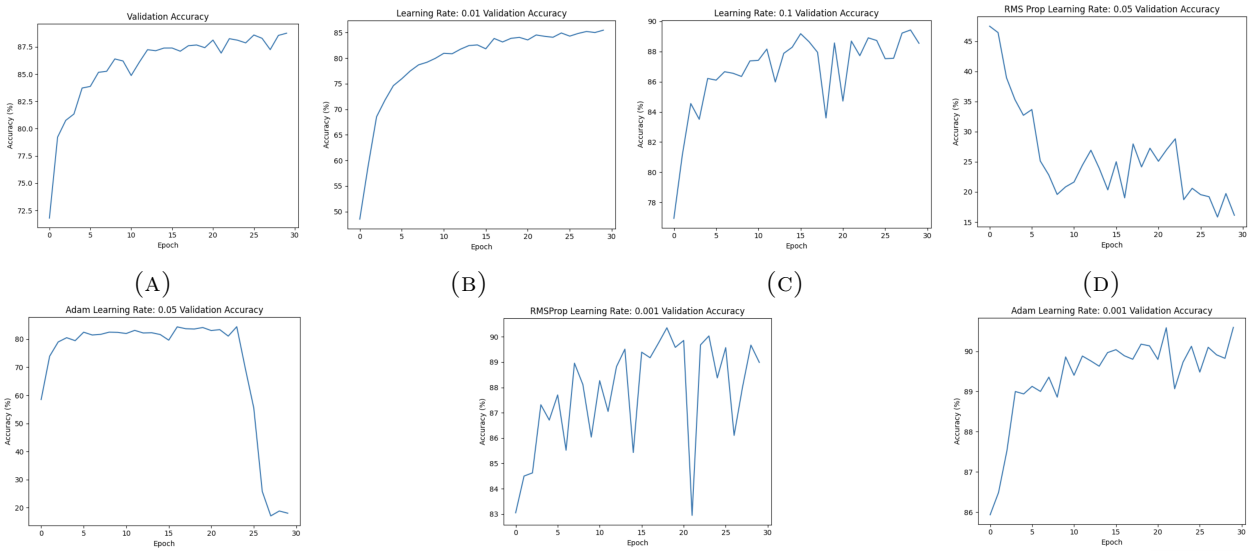
The baseline model trained over 30 epochs, used a learning rate of 0.05, had an input dimension of 784, an output dimension of 10, hidden dimension of 512, and 2 hidden layers. It used an SGD optimizer.

Question 4a: For this question we observed the effect of different optimizers on our training loss, validation accuracy, and test accuracy.

	Baseline	**SGD 0.01**	**SGD 0.1**	**RMS 0.05**	**Adam 0.05**	**RMS 0.001**	**Adam 0.001**
Final Train Loss	0.2584	0.4033	0.1981	2.1160	2.1156	0.0903	0.0727
Final Val. Accuracy	88.62%	84.78%	88.54%	16.13%	18.02%	88.99%	90.59%
Test Accuracy	87.44%	83.68%	87.72%	16.80%	17.95%	88.73%	89.16%

TABLE 1. When fixing the learning rate across every single optimizer, the RMS and Adam optimizers did very poorly. The model that had the best training loss and validation accuracy was the baseline model, and the model with the best test accuracy was the model that used a stepsize of 0.1. However, to make the comparison more reasonable, I tested the RMS and Adam optimizers on a more reasonable step size of 0.01. After doing this, the model with the Adam optimizer performed the best in train loss, validation accuracy, and test accuracy.

These visualizations show the training loss over epoch for different optimization configurations. I did not include the graphs of epoch vs training loss, as most of them looked relatively similar in terms of stability (except for Adam and RMS Prop with 0.05 learning rate which had a sharp drop).



(A) (B) (C) (D)

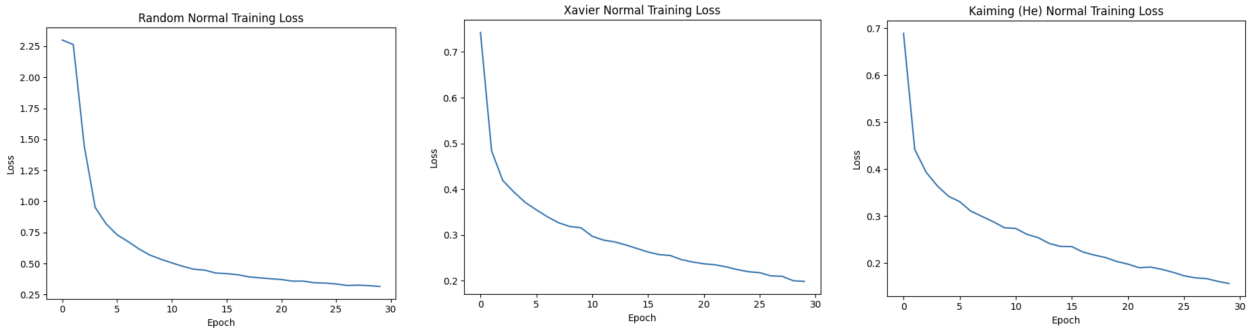
(E) (F) (G)

FIGURE 1. The optimizier that appears to be the most stable across the 3 is SGD. Regardless of learning rate, its deviations are relatively stable. When Adam and RMS Prop's learning rates are higher they perform very poorly for validation accuracy. While using a smaller learning rate on RMS Prop increases its validation accuracy, its performance is highly unstable. The model that seems to meet the middle ground on stability and accuracy is Adam, which is also corroborated by our results from table 1.

Question 4b: For this question we will compare different initializers.

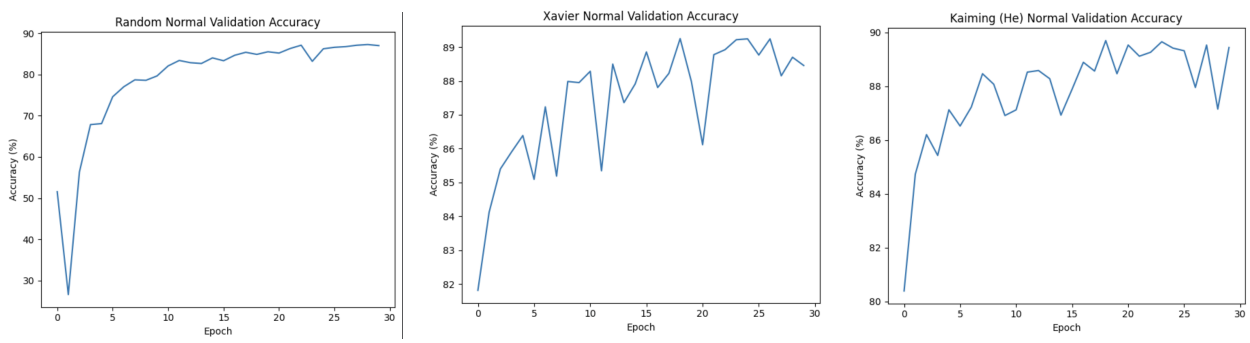
	Random Normal	**Xavier Normal**	**Kaiming Uniform**
Final Train Loss	0.3149	0.1985	0.1562
Final Val. Accuracy	87.03%	88.46%	89.44%
Test Accuracy	85.50%	87.73%	88.33%

TABLE 2. The model that performed the best in all categories is the model that uses a Kaiming (He) Uniform initializer. The model that performs the worst is the Random Normal model. This makes sense as Kaiming (He) combines the best aspects of both Xavier Normal and Random Normal in its performance.



(A) (B) (C)

FIGURE 2. The convergence rates of all graphs seem relatively stable, the only difference is the starting and ending loss value.



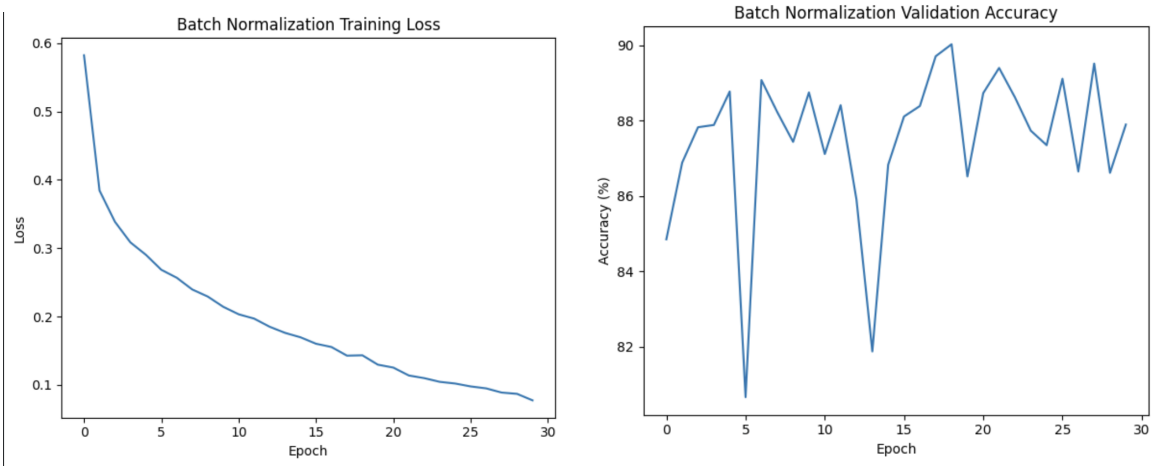
(A) (B) (C)

FIGURE 3. Here we can see that the model that has the highest validation accuracy while meeting the middle ground for stability is the model using Kaiming (He) initialization.

Question 4c:

	Baseline	**Batch Normalization**
Final Train Loss	0.2584	0.0774
Final Val. Accuracy	88.62%	87.90%
Test Accuracy	87.44%	86.84%

TABLE 3. The results of batch normalization seem interesting, as the train loss is significantly better than the baseline model, while the baseline model's test and validation accuracy is higher. I suspect that the use of normalization made our model overfit to the data, as that normally leads to great training performance and low test and validation performance.



(A) (B)

FIGURE 4. While the training loss stability has not been affected much (even though the final value is lower), when comparing the trend in validation accuracy over epochs with the baseline, the stability is significantly worse. This might be because of the models sensitivity to new data because it overfit.

5. Summary and Conclusions

In this project, I implemented a fully connected neural network which can be used as a classifier for the FashionMNIST dataset. I learned through my experimentation on the parameters that the best optimizer is the Adam optimizer with a very low learning rate of 0.001. If we use a learning rate of 0.05, the baseline model has the best validation set performance, while the model with a learning rate of 0.1 performed the best on the test set, so that should be used. The initialization method that was the best was using Kaiming (He) Uniform, and using batch normalization caused the model to overfit.

Acknowledgements

The author is thankful to Prof. Frank for useful discussions about Neural Networks, initialization, and optimization.

References

[1] C. O. da Costa-Luis, S. K. Larroque, K. Altendorf, H. Mary, richardsheridan, M. Korobov, and N. Yorav-Raphael. tqdm: A fast, extensible progress bar for python and cli, 2023.

[2] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.

[3] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[4] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[6] TorchVision maintainers and contributors. Torchvision: Pytorch's computer vision library. `https://github.com/pytorch/vision`, 2016. GitHub repository.